Not the Metric You Report: Leakage-Audited Landmine Surface Detection, and Why mAP_{50} Cannot Choose an Operating Point

Anonymous Authors
Preprint. Under review.

***Abstract*—Detection benchmarks in aerial and remote-sensing settings are usually ranked by mean average precision (mAP). We show that on a small, single-class landmine surface-indicator dataset this ranking is uninformative for deployment: a 3.0 M-parameter detector and an 11.1 M-parameter detector differ by less than the seed-to-seed spread in test mAP_{50} (0.979 vs. 0.977 ± 0.003), yet their recall-first operating points differ by a factor of 6.2 in false positives per image (7.266 vs. 1.165). We argue that the meaningful object of study is the operating point, not the summary curve, and we build the evaluation protocol around that claim.**

The study rests on two methodological contributions that require no new architecture. First, we audit a widely reused landmine image corpus and find that its published split leaks: 76 Roboflow base-name groups span splits and 63.8% of evaluation images have a near-duplicate in training, consistent with augmentation applied before splitting. We rebuild the split group-wise, apply a two-method leak filter (perceptual hash plus pixel MSE) with manual review of every candidate, and remove 30 evaluation images. The frozen manifest (3,907 / 181 / 189 images; digest `748322d6...`) contains no exact duplicate, no base-key collision, and no confirmed near-duplicate leak.

Second, we measure what the leak was worth. A detector trained on the leaky split scores 0.994 mAP_{50} on the full clean test set against 0.974 for a clean-trained model, but on the 49 contamination-free test images the two are tied (0.995 vs. 0.995). This shows a small mAP difference on the contamination-free subset, but the operating points were not threshold-matched, so we do not infer that leakage has zero effect on deployment behaviour.

We then evaluate a MobileNetV3-Large second stage over detector crops on five detector seeds. The two-stage arm improves precision over a baseline matched to the two-stage validation recall at three of five seeds ($\Delta P = +0.128$, $+0.144$ and $+0.126$, bootstrap intervals excluding zero), is indistinguishable at the fourth ($\Delta P = -0.037$, interval $[-0.077, +0.013]$), and is *worse* at the fifth ($\Delta P = -0.049$, interval $[-0.091, -0.002]$). A confidence interval over the five per-seed paired differences spans zero ($+0.062$ $[-0.058, +0.182]$), so we do not claim a mean improvement. Under a max-F_2 rule a val-tuned single-stage threshold reaches the same F_2 with higher precision and fewer false alarms ($P = 0.976$ vs. 0.945). We report the effect as detector-dependent and interpret the threshold-transfer explanation as a hypothesis consistent with these five seeds, not as an identified mechanism; the seed that regresses is traced to a validation-only selection rule choosing a near-inert classifier, not to a classifier failure. The Grad-CAM results are descriptive evidence rather than proof of correctness or context use. All experiments were run on one 8 GB laptop GPU within a $\sim$30 h budget; we state explicitly what that budget forbade. Finally, we document an evaluation gap that the leakage literature does not name: the checkpoint deployed in the accompanying system is not any checkpoint evaluated here—different backbone, different split—and neither repository records the transition. We call this *artifact* drift, and we give the per-artifact manifest that prevents it.

***Index Terms*—landmine detection, object detection, dataset leakage, operating point, evaluation protocol, YOLOv8**

I. Introduction

A landmine detector that is evaluated with mean average precision is being asked the wrong question. Mean average precision integrates performance over every confidence threshold and every recall level, then reduces the result to one number. A demining team does not operate a curve; it operates one threshold, on one day, with one tolerance for false alarms. Two systems can share a curve and behave completely differently at the threshold that will actually be used. This paper is an extended argument, with measurements, that the operating point—and not the summary curve—is the object that should be reported, and that the usually unglamorous work of auditing the split is what makes any such comparison believable.

We study single-class detection of *surface* landmine indicators from downward-looking aerial imagery. The scope limitation is physical and we state it first: a visible-light or colour camera cannot see a buried object. What it can see is a mine lying on the surface, a partially exposed casing, or a surface marker. Claims in this paper are therefore claims about visible surface objects, not about buried ordnance, and nothing here is a field-validated clearance system. The task is nonetheless a useful stress test for detection methodology, because the class is rare, the imagery is heterogeneous, and the published data reuse is heavy.

What we found.: Three results organise the paper.

1) **The split leaks, and the leak is measurable.** The combined landmine corpus contains 76 filename groups (Roboflow base names) that span train and evaluation splits, and 63.8% of validation and test images have a perceptual-hash neighbour in training. This is the signature of augmentation applied before splitting: the same scene, re-exported, lands on both sides of the boundary. After rebuilding the split group-wise and filtering twice, 30 evaluation images are removed as true leaks. A detector trained on the leaky split scores 0.994 mAP_{50} on the full clean test set, against 0.974 for a clean-trained detector. On the 49 test images that are contamination-free for both, the two are tied at 0.995 and 0.995. The leak inflated the number, but by less than the

folklore predicts—a negative result, and one that would have been invisible without the contamination-free subset.

2) **$\mathbf{mAP}_{50}$ does not settle the deployment question.** A YOLOv8s detector over five seeds reaches 0.976 ± 0.003 test mAP_{50}; YOLOv8n over three seeds reaches 0.979 ± 0.003. Although the n point is slightly higher on test, validation mAP favors s (0.955 ± 0.006 vs. 0.934 ± 0.002), and the test intervals overlap almost completely. At the recall-first operating point chosen on validation, the n seeds produce 7.27 ± 4.42 false positives per image against 1.17 ± 0.86 for s. We present this as a case study in threshold transfer and operating-point reporting, not as a reliable model-ranking reversal.
3) **The two-stage gain is detector-dependent, and we can say why.** A MobileNetV3-Large classifier over detector crops improves precision over a recall-matched single-stage baseline at three of five detector seeds ($\Delta P = +0.128$, $+0.144$ and $+0.126$, paired bootstrap intervals excluding zero). At a fourth it is indistinguishable ($\Delta P = -0.037$, interval $[-0.077, +0.013]$) and at the fifth it is *worse* ($\Delta P = -0.049$, interval $[-0.091, -0.002]$). A confidence interval over the five per-seed paired differences spans zero ($+0.062$ $[-0.058, +0.182]$), so the mean gain is not established. Under a max-F_2 rule, a single confidence threshold tuned on validation beats the two-stage pipeline outright ($P = 0.976$ vs. 0.945). The pattern is consistent with the classifier helping when the validation-selected detector threshold transfers poorly to test, but five seeds with shared components do not identify that mechanism. Where the baseline threshold transfers well, the second stage has nothing to add; where the selection rule picks a near-inert classifier, it actively subtracts. Reporting a pooled mean would have hidden this entirely; we report the per-seed structure instead.

Why this is worth publishing.: None of the three findings required a new architecture, a large GPU, or a new dataset. They required (i) reconstructing the split from scratch, (ii) refusing to average over seeds when the seeds disagree structurally, and (iii) measuring at the operating point rather than at the summary. The total compute was one 8 GB laptop GPU and roughly 30 hours. We think this is a realistic and under-reported budget for applied detection work, and we document what it forbade (Section X) rather than implying broader coverage.

Contributions.:

- A reproducible leakage audit of a publicly derived landmine corpus: an operational leak definition, a two-method filter with a manually validated MSE cut, a frozen manifest, and an accounting of all 30 removals (Section IV).
- A direct leaky-versus-clean comparison on a contamination-free subset, including the unfavourable result that the mAP difference on a small contamination-free subset is small, while threshold-matched operating-point effects remain unmeasured (Section IV-D).
- An operating-point evaluation protocol—thresholds chosen on validation only, applied once to test, with paired image bootstrap and source-cluster intervals—and a case study showing why a single mAP-based ranking can be insufficient (Section VII).
- A per-seed analysis of a two-stage detector-plus-classifier pipeline that localises a failure rather than averaging it away, together with a max-F_2 analysis showing when the second stage is not worth its cost (Section VII-D).
- A Grad-CAM audit used as a sanity check on evidence, with an explicit separation between attribution and correctness, showing that the image-level classifier's accuracy is not accompanied by object-level evidence (Section VII-G).
- A documented instance of *artifact* drift: the checkpoint behind the accompanying deployment project differs from every checkpoint evaluated here in both backbone and split, and neither repository records the transition. We give the artifact-level account that would have prevented it (Section IX).

Section II positions the work; Section III describes the corpus and its provenance; Section IV the audit; Sections V–VI the models and protocol; Section VII the results; Section VIII the discussion; Section IX the deployment architecture and the artifact-drift finding; Sections X–XI limitations and ethics. Code, weights, tables and the frozen split manifest accompany the paper (Section XII).

II. Related Work

Landmine and UXO detection.: The physical sensing literature is large and predates deep learning by decades: metal detectors, ground-penetrating radar, infrared and multispectral sensing, and combinations thereof [1, 2]. What matters for this paper is a distinction that the computer-vision literature often blurs. Subsurface sensing asks whether a buried object is present; visible-band aerial imaging asks whether a surface object or marker is visible. The two tasks have different base rates, different failure modes, and different false-alarm costs. We therefore restrict every claim to the visible-surface task and treat buried-object performance as out of scope rather than as a gap to be estimated. The humanitarian scale of the problem is documented annually by the Landmine Monitor [3]; we mention it to motivate careful evaluation, not to claim operational utility.

Detectors.: The YOLO family [4, 5] remains the default for small-object aerial detection because of its latency at moderate resolution. Alternatives include anchor-free and transformer-based detectors [6–8] and two-stage region proposals [9], with focal loss [10] addressing foreground imbalance. Our contribution is not a detector. We deliberately use the most standard available baseline, because the point is that evaluation choices dominate architecture choices at this scale. The lightweight classifier in our second stage is MobileNetV3-Large [11] (3,119,535 parameters, torchvision `mobilenet_v3_large` with ImageNet weights), chosen for cost rather than novelty.

Leakage and reproducibility.: That machine-learning results can be inflated by information leaking across train and evaluation splits is now well documented, both as a general

methodological failure [12–14] and in specific benchmarks [15, 16]. The canonical defence is group-aware splitting, so that no group—scene, patient, site, or source document—spans the boundary; and once the split is fixed, the estimator used to turn it into a number is itself a choice, since cross-validation and bootstrap schemes of the same data do not agree with each other [17]. In image corpora, groups must be recovered from pixels when filenames are unreliable: perceptual hashing [18] proposes near-duplicate candidates, and a pixel-space distance such as MSE or SSIM [19] separates true re-exports from terrain-texture coincidences. Our audit follows exactly this two-stage pattern, and we report the threshold at which the two methods disagree. Publicly shared detection datasets frequently originate as re-exports of the same source collection, which is why base-key collisions matter independently of pixels.

Evaluation beyond mAP.: The mismatch between average precision and deployed behaviour is an old observation in detection. Hoiem et al. [20] argued for error diagnosis over single scores; Dollár et al. [21] popularised log-average miss rate at a fixed false-positive-per-image budget precisely because average precision hides the low-false-alarm regime. Our protocol adopts false-positives-per-image at a recall-first operating point as the primary quantity, and reports mAP only as a secondary, comparable-to-the-literature number. We also follow the recommendation to report intervals rather than point estimates: paired image bootstrap for comparisons on the same images [22], and source-cluster resampling where the effective sample size is the number of acquisition sources, not the number of images. Agreement statistics use Cohen's κ [23] for image-level decisions. Dataset and model documentation follow the datasheet and model-card conventions [24, 25].

Attribution methods.: Grad-CAM [26] and its variants [27] are widely used to argue that a model "looks at the right thing". The evidence that saliency maps can be insensitive to model parameters [28], and that deletion-style tests are the appropriate falsification [29], means attribution should be treated as a diagnostic and never as proof of correctness. The Clever Hans literature [30] is the cautionary case: a classifier can be accurate for the wrong reason. We use Grad-CAM in that falsification role only, and we report where it fails.

III. Data

A. Corpus and task

The corpus pools five public Roboflow Universe landmine-detection exports, identified here by source id: `ds0`, `ds1`, `May_lwir`, `Jan_lwir`, and a small derived set folded into `ds0`. Each image carries one class, `landmine`, and axis-aligned boxes. There is no mine-type label; the task is presence and localisation of a visible mine-like object. Table I gives the frozen split.

The class is heavily imbalanced in the detection sense and, more importantly, heterogeneous in acquisition. Table II shows the source/split shift that dominates the results later. `Jan_lwir` images average 7.7 boxes per test image; `ds0` and `ds1` average about one. A model that learns "many objects" from `Jan` and applies it to `ds0` will generate false positives by construction, and vice versa. This is not a nuisance to be averaged away: it is the principal source of variance in the test set, and we return to it in Section VII-B.

B. Two content defects, disclosed

The `lwir` files are not thermal.: Sources named `May_lwir` and `Jan_lwir` suggest long-wave infrared imagery. They are not. Mean HSV saturation on a 24-image sample is 0.38, against 0.25 for the `ds0` RGB reference, and the images carry visible marker pins. They are colour RGB photographs. We keep them, because removing them would drop the only multi-object source and make the task artificially easy, and we disclose the consequence: a model could exploit marker pins as a shortcut. A twelve-crop zoom check around test mines found no consistent marker, paint, or flag cue, but we cannot rule out a subtler one, and we do not make thermal claims anywhere in this paper.

Two test labels are objectively wrong.: Manual review identified two test boxes that are not on mines: `ds0_0801` annotates a tree trunk and `ds0_0883` annotates a fence. Both were confirmed independently by a Grad-CAM energy of exactly zero (Section VII-G). We remove them from ground truth and report results both with and without the correction (Table III). No training labels were found to be wrong, so no retraining was performed.

C. Provenance and redistribution

The source archives were obtained as public exports, but their licence terms were not recorded at download time and the original project URLs are no longer available to this project. We therefore do not redistribute any image, label file, or crop. What is released is the split manifest with a SHA-256 digest, the removed-leak list with reasons, the audit CSVs and montages, the trained weights, and the code. Anyone holding the original archives can reconstruct the exact splits byte-for-byte from the manifest. Full detail is in `DATA_PROVENANCE.md`; the weights carry the upstream Ultralytics AGPL-3.0 terms (Section XII). We regard the missing licence record as a defect of this study and state it rather than inferring a permissive licence.

IV. Leakage Audit

A. The original split fails

The first audit pass over the pooled corpus returned a clear failure. Of the evaluation images, 63.8% had a training image within perceptual-hash Hamming distance 2, and 76 Roboflow base names appeared in more than one split (train∩valid 37, train∩test 31, valid∩test 10). The mechanism is augmentation before splitting, present in the upstream generation scripts. The test split also contained zero negative images, so false-alarm rate was undefined. Figure 1 is the evidence for the stronger of the two findings, and we describe what it shows rather than asking the reader to accept an aggregate.

We therefore discarded the split entirely rather than attempting to patch it.

TABLE I

FROZEN DATASET LANDMINE_COMBINED_V2. "NEG" COUNTS IMAGES WITH AN EMPTY LABEL FILE (DELIBERATE NEGATIVES), NOT MISSING LABELS; THERE ARE ZERO MISSING LABELS. MANIFEST DIGEST 748322D6...C80DE.

split	images	instances	neg	sources
train	3,907	11,550	20	Jan 100, May 776, ds0 2,589, ds1 442
valid	181	419	7	Jan 42, ds0 138, ds1 1
test	189	253	5	Jan 10, ds0 151, ds1 28

TABLE II

SOURCE AND SPLIT COMPOSITION. THE TEST SET IS DOMINATED BY DS0 IMAGES (151 OF 189) BUT BY JAN BOXES (77 OF 253).

split/source		images	boxes	boxes/img	empty rate
test	Jan_lwir	10	77	7.70	0.00%
test	ds0	151	148	0.98	1.99%
test	ds1	28	28	1.00	7.14%
train	Jan_lwir	100	740	7.40	0.00%
train	May_lwir	776	7,706	9.93	0.00%
train	ds0	2,589	2,643	1.02	0.77%
train	ds1	442	461	1.04	0.00%
valid	Jan_lwir	42	287	6.83	0.00%
valid	ds0	138	131	0.95	5.07%
valid	ds1	1	1	1.00	0.00%

TABLE III

EFFECT OF REMOVING TWO OBJECTIVELY WRONG TEST BOXES (NO RETRAINING). INTERVALS IN THE SOURCE ARTIFACT ARE UNCHANGED; THE SHIFT IS A GROUND-TRUTH CORRECTION, NOT A MODEL CHANGE.

arm	metric	before	after
C (two-stage)	box recall	0.9644	0.9721
C (two-stage)	image recall	0.9891	1.0000
C (two-stage)	FP/image	0.153	0.153
A (YOLO deploy)	box recall	0.9763	0.9841
B (YOLO matched)	box recall	0.9723	0.9801

B. Operational leak definition

A validation or test image x is a leak if any training image shares any of:

1) the exact MD5 digest;
2) the Roboflow base key (filename with the `*.rf.<md5>.jpg` export suffix stripped);
3) the same source id *and* a 64-bit perceptual hash [18] within Hamming distance 2 under all eight dihedral transforms, *and* a 32×32 RGB mean-squared error below 900.

Condition 3 is a conjunction for a reason. Perceptual hash alone over-fires on repetitive terrain: grass and dirt produce Hamming collisions that are not the same scene. The MSE term separates the two populations, and manual review sets the cut. Every Hamming-≤ 2 candidate—146 pairs across two rounds—was rendered side by side and inspected. Confirmed same-scene pairs occurred at $\mathrm{MSE} \leq 817$; terrain-texture coincidences at $\mathrm{MSE} \geq 958.6$. The gap is narrow but unambiguous on montage, and the cut at 900 sits inside it.

C. Rebuilding and freezing

Groups were formed from base key, capture session, and same-source hash edges, yielding 1,048 groups, then assigned whole to a split. Two filtering passes removed 30 evaluation images: 13 at $\mathrm{MSE} \leq 100$, then 17 more at $\mathrm{MSE} \leq 900$, including a cross-folder duplicate (`ds0_0043`↔`ds1_0043`). The split is 19 validation and 11 test images lighter as a result. The residual 99 of 370 evaluation images with a Hamming-≤ 2 training neighbour all have $\mathrm{MSE} \geq 958.6$ and were verified as texture false positives. True leak rate: $0/370$.

D. What the leak was worth

An audit is only useful if it changes something. We therefore trained a YOLOv8s detector on the *old* split (the "leaky" model) and compared it with the clean-trained seed-0 detector on the clean test set. This is the natural before/after experiment, and it has a trap: 140 of 189 clean test images (74.1%) share a v2 group with an image the leaky model trained on. Scoring the leaky model on the full test set therefore measures memorisation.

The results are in Table V. On its own split the leaky model reaches 0.970 mAP_{50}, but that split has zero negatives, so false-positive rate is undefined and the number cannot be compared with anything in this paper. On the full clean test set it reaches 0.994, comfortably above the clean model's 0.974. On the 49 contamination-free images the two are tied: 0.9950 against 0.9947 mAP_{50}, and 0.725 against 0.701 $\mathrm{mAP}_{50\text{–}95}$. The 49-image subset is small and the comparison is therefore low-powered. The mAP difference is small on this subset, but the operating-point values are not comparable because the two models use different validation-selected confidences; we do



Fig. 1. Base-key collisions between the test and train splits of the original corpus, drawn from the first audit pass. Each pair is one test image (left) and the train image (right) that shares its Roboflow base name, i.e. the same source file re-exported into both splits. The pairs are the same physical scene: the same mine placement, terrain texture and shadow, differing only in crop, colour jitter and export bytes. These are re-exports, not distinct photographs, and no threshold choice is involved in exhibiting them. Twenty of the 76 colliding base names are shown.

not infer that leakage has zero effect on deployment behaviour from this comparison.

This is a negative result and we treat it as one. The literature on leakage [12, 15] prepares one to expect a large inflation. Here the clean and leaky models converge on the leak-free images, which suggests that for this task the leaked material mostly taught the model the same thing the clean data does. That is not a reason to skip the audit—the old split remains unusable, because its selection split was contaminated and its test split had no negatives—but it is a reason to report the size of the effect honestly rather than using the audit as a rhetorical amplifier. It also has a practical implication: an audit that only reports "we fixed leakage" without measuring the contamination-free delta leaves the reader unable to judge whether the fix mattered.

V. Method

A. Detector

We use Ultralytics YOLOv8 [5] at two depths, `yolov8s` and `yolov8n`, both with a single-class head. Models are initialised from COCO-pretrained weights and fine-tuned for 75 epochs at 640×640, batch 8, with the Ultralytics default augmentation recipe and early-stopping patience 20. The `s` model is trained with seeds $0, 1, 2, 3, 4$; the `n` model with seeds $0, 1, 2$, as a smaller size point. All runs use the same pinned stack (Ultralytics 8.4.117, PyTorch 2.6.0+cu124) on one RTX 4070 Laptop GPU with 8 GB of memory.

Checkpoint selection is done on validation. For each seed we keep the checkpoint (last or best) that gives the better validation operating point, then choose the confidence threshold on validation and apply it unchanged to test. No test quantity influences any choice. This is the only point in the protocol where discipline costs something: for some seeds the validation-optimal threshold transfers badly to test, and we do not fix that after the fact (Section VII-D).

B. Operating points

For a set of detections, precision P, recall R and false positives per image $\mathrm{FP/img}$ are all functions of a confidence threshold. We define two operating points on validation and carry both to test:

- **Recall-first (OP_{97}).** The largest confidence whose validation box recall is at least 0.97. This is the operating point a demining workflow would plausibly choose: a missed mine is a safety event, a false alarm is an analyst cost. It is the primary point in all tables.

TABLE IV
LEAKAGE AUDIT OUTCOME FOR THE FROZEN SPLIT. "RESIDUAL CANDIDATES" ARE HAMMING-≤ 2 PAIRS REJECTED BY THE PIXEL-MSE TEST AND MANUAL REVIEW.

quantity	value
Images (train / valid / test)	3,907 / 181 / 189
Instances	11,550 / 419 / 253
Negatives	20 / 7 / 5
Exact cross-split duplicate groups	0
Base-key collisions (train-valid / train-test / valid-test)	0 / 0 / 0
Residual near-duplicate candidates	99 (all MSE ≥ 958.6)
Evaluation images removed as leaks	30 (19 valid, 11 test)
Confirmed true leak rate	$0/370 = 0.0\%$
Manifest SHA-256	`748322d6...c80de`

TABLE V
LEAKY-VERSUS-CLEAN COMPARISON. THE LEAKY MODEL'S OPERATING CONFIDENCE (0.065) WAS CHOSEN ON THE OLD VALIDATION SPLIT; THE CLEAN MODEL'S (0.02) ON THE NEW ONE. P AND FP/IMAGE ARE THEREFORE NOT THRESHOLD-MATCHED AND MAP IS THE FAIR COLUMN.

model	evaluation set	mAP_{50}	$\text{mAP}_{50\text{–}95}$	P	FP/img
leaky v1	v1 test (own split)	0.9697	0.6436	0.834	n/a (0 neg)
leaky v1	v2 test, full (74.1% contaminated)	0.9940	—	0.857	0.222
clean v2 s0	v2 test, full	0.9741	0.6924	0.427	1.751
leaky v1	v2 test, contamination-free ($n = 49$)	0.9950	0.7250	0.863	0.204
clean v2 s0	v2 test, contamination-free ($n = 49$)	0.9947	0.7010	0.354	2.347

- **High-precision (OP_{95}).** The largest confidence with validation recall at least 0.95, used only for the image-level comparison.

mAP is computed in the usual way at a low confidence floor (0.001) and reported for comparability with the literature. It plays no role in any selection.

C. Two-stage pipeline

The second stage is a MobileNetV3-Large classifier [11] applied to detector crops. We keep the torchvision `mobilenet_v3_large` convolutional trunk with ImageNet weights and replace the classification head with a 128-unit bottleneck (global average pool, linear 128 with ReLU, dropout 0.3, linear 1), yielding 3,119,535 parameters in total. Each detection box is expanded by 20% on each side, cropped, and resized to 224×224; the classifier outputs a single sigmoid score. Training crops are 11,550 positives and 15,411 negatives, the latter comprising random background, 3,730 detector false positives mined at confidence 0.05 from an epoch-20 checkpoint, and 20 empty-frame samples. Positives and negatives are sampled 1:1 per epoch; the classifier is fine-tuned end-to-end for 30 epochs at learning rate 2×10^{-5} with an unweighted binary cross-entropy loss. No class weighting is needed here, because the 1:1 crop sampling already balances the batch; this is the opposite of the image-level baseline, which is trained on the raw 3,887:20 positive-negative split and therefore requires per-sample weights $[97.7, 0.50]$ (Section VII-C). The 1:1 ratio and the unfrozen-late-layers recipe were chosen on validation, and a five-variant ablation is reported in Section VII-F.

We compare three arms, all evaluated on test exactly once with validation-frozen parameters:

- **A (deploy).** YOLO at its validation-chosen OP_{97} confidence.
- **B (matched recall).** YOLO at the confidence that matches arm C's *validation* recall. This compares the two-stage arm with a YOLO-only baseline at the same validation recall and isolates the classifier's contribution from a trivial threshold change.
- **C (two-stage).** YOLO proposals at a fixed low confidence (0.05) are routed through the classifier; a proposal is kept if it passes either a *protection* rule (detector confidence $\geq P$) or the classifier threshold τ. P is swept over $\{0.4, 0.6, 0.8, \infty\}$ on validation; for each P we take the largest τ whose validation recall is within 0.5 points of stage-1 recall, then choose the pair with the lowest validation FP/image. Four of the five seeds select $P = 0.4$, with $\tau = 0.27/0.23/0.22/0.13$ for seeds $0/1/2/3$. Seed 4 selects $P = 0.8$ with $\tau = 0.03$, the one place where the rule leaves the $P = 0.4$ regime; Section VII-D shows that this choice is what makes arm C regress at that seed, and we report it rather than overriding it.

The protection rule matters. Without it the classifier can veto a high-confidence detection, and on this corpus that costs recall on the multi-object `Jan` source. With $P = 0.4$ the pipeline is allowed to suppress only low-confidence proposals, which is the regime where the classifier's crop evidence is useful. The rule's weakness is the opposite case: a high P with a very low τ leaves almost every proposal protected, so the classifier is present but inert while still satisfying the recall constraint.

Figure 2 shows the three arms as data flow. Arms A and B are scoring rules applied to the same detector run; only arm

C adds parameters. The protection branch is a bypass around the classifier, not a gate in front of it, which is why a high P can make arm C collapse into arm A without changing any classifier weight.

D. Statistics

All test comparisons use the same 189 images, so we resample images, not detections. For a paired comparison between two arms we draw 5,000 bootstrap replicates of the image set [22] and report the 95% percentile interval of the difference. For single-arm uncertainty we additionally report a source-cluster bootstrap over the three test sources, and a leave-one-source-out analysis; with three clusters these intervals are wide by construction, and we present them as such rather than as precise estimates. Image-level agreement uses Cohen's κ [23] alongside balanced accuracy, because the test set has only 5 negative images and raw accuracy is dominated by the positive class.

E. Attribution audit

We compute Grad-CAM [26] on the neck feature maps feeding the detection head, targeting the top pre-NMS class score, and on the last convolutional block of the classifier. Because the top pre-NMS anchor usually sits on the stride-8 head, the largest stride with a non-zero gradient is used; this deviates from the common stride-32 choice and is documented with the code. We report four quantities: the share of CAM energy inside the ground-truth box (and its enrichment over the box's area share), whether the CAM peak falls inside the box, the drop in detection score when the box is masked versus when a matched background region is masked, and a head-to-head comparison of detector and classifier saliency on the same crops. Every sentence about these numbers is phrased as evidence about *where the model looked*, never about whether the prediction is correct.

VI. Experimental Protocol and Reproducibility

Frozen rules.: Five rules governed every run, and we report violations as well as compliance. (i) Thresholds are chosen on validation only. (ii) Every evaluation that touches test is logged, numbered, and justified in advance. (iii) Every number in this paper is produced by a committed script reading a committed JSON or CSV. (iv) The original dataset and the pre-audit runs are never modified. (v) Test labels are corrected only when objectively wrong, with before/after numbers reported.

Test-touch accounting.: The full log contains 25 numbered touches. Five are large: the seed-0 frozen-protocol detector evaluation, the seed-0 and seed-1/2 two-stage evaluations, the corrected detector-aligned two-stage re-scoring, and the four-seed extension of the two-stage pipeline. The remainder are the per-seed detector evaluations, the MNv3 baseline evaluations, the Grad-CAM passes, the stratified Grad-CAM re-cut, and the four analysis re-reads recorded in the extended-analysis block; each is sanctioned by the protocol before execution. We also disclose a deviation: the upstream training script auto-evaluates test at confidence 0.25 at the end of training, and this behaviour could not be disabled by flag. It is recorded as D017. No threshold, checkpoint, or architecture choice references those numbers; they appear in no table here. The frozen-protocol evaluation is a separate, single pass.

Correction we had to make.: An earlier version of the two-stage result reused the seed-0 detector for all three seeds, producing a tight mean of 0.879 ± 0.018. The arms A and B records were byte-identical across seeds, which is the tell. Re-running the pipeline with each seed's own detector gives $0.894/0.825/0.724$ and a standard deviation of 0.086. The tight interval was an artifact of a shared component. With the two additional detector seeds the five-seed mean is 0.782 ± 0.085. We report the corrected numbers throughout and flag the earlier figure as withdrawn.

Compute.: All training and evaluation ran on one NVIDIA RTX 4070 Laptop GPU with 8 GB of memory, under Windows, within an approved budget of roughly 30 GPU-hours. Detector training took 50–85 minutes per seed; the second-stage classifier 30 epochs per run; the ablation matrix five classifier retrains. Evaluation uses 640×640 input, NMS IoU 0.7, `max_det` 300, FP32, no test-time augmentation. Latency figures are PC GPU FP32 medians over 50 repetitions after 10 warm-up iterations, and are *not* edge-device numbers; no Jetson or TensorRT measurement was performed.

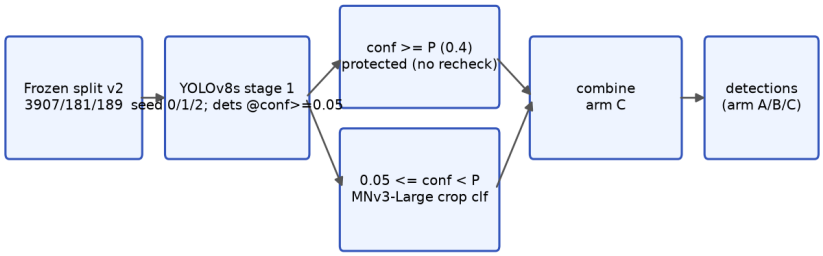
What the budget forbade.: We did not train YOLOv8m or YOLOv8l on the clean split, did not evaluate a second detector family (RT-DETR or Faster R-CNN), did not run grouped cross-validation, did not re-mine hard negatives on data the detector had not seen, and did not retrain after the test-label correction. Each omission is a real limitation and is listed in Section X. We prefer to name them than to let a single-seed or single-family result stand in for a claim it cannot support.

Artifacts.: The repository contains the code, the released detector checkpoints (`yolov8s` seeds 0–2, each with `best.pt`, `last.pt` and an ONNX export), the three stage-2 checkpoints, the frozen split manifest and its digest, all evaluation JSONs, the generated tables, and the figures. The seed-3 and seed-4 `yolov8s` runs and the `yolov8n` seeds 0–2 were trained on the same frozen split and their evaluation JSONs are committed, but their weight files are not part of the release tree. Regenerating the tables requires no GPU and no dataset: the committed JSONs are sufficient, and the table script fails loudly if a required field is missing rather than silently substituting a value.

VII. Results

A. Size sweep: mAP saturates, the operating point does not

Table VI reports both depths. The mAP columns are almost flat: 0.976 ± 0.003 for `yolov8s` over five seeds and 0.979 ± 0.003 for `yolov8n` over three. On validation the gap is larger (0.955 ± 0.006 vs. 0.934 ± 0.002), but the two intervals do not overlap and the direction favours the larger model, which means we cannot claim the smaller model is better on this evidence. Neither can we claim it is worse: the test interval for `n` sits entirely inside the `s` interval, so mAP_{50} does not resolve the pair.



al cache is conf>=0.05 (one cache per detector seed 0/1/2); crop = box + 20% context, 224x224, sigmoid score; tau and P chosen on val only. Arms A/B are scored on the same detector runs with their own va

Fig. 2. The two-stage pipeline as data flow. The detector proposes at a low confidence (0.05); a proposal either bypasses the classifier when its detector confidence is at least the protection threshold P, or is scored by the MobileNetV3-Large crop classifier and compared against τ. Both P and τ are chosen on validation. Arms A and B are the same detector run scored by a single confidence, so the schematic differs between arms only in where the final comparison happens, not in the features computed.

TABLE VI
SIZE SWEEP ON THE FROZEN TEST SPLIT. BOTH ROWS ARE MEAN ± STD OVER SEEDS (5 FOR `s`, 3 FOR `n`). "OP" IS THE VALIDATION-CHOSEN RECALL-FIRST OPERATING POINT ($R \geq 0.97$ ON VALIDATION), APPLIED UNCHANGED TO TEST. LATENCY IS PC GPU FP32, 640×640, MEDIAN OF 50 RUNS.

model	params	GFLOPs	MB	ms	val mAP_{50}	test mAP_{50}	R	P	FP/img
yolov8s	11.1 M	28.6	21.5	6.5	0.9546 ± 0.0064	0.9763 ± 0.0033	0.9771 ± 0.0076	0.579 ± 0.181	1.165 ± 0.863
yolov8n	3.0 M	8.2	6.0	6.9	0.9338 ± 0.0017	0.9787 ± 0.0025	0.9855 ± 0.0023	0.185 ± 0.095	7.266 ± 4.421

The operating-point columns are not flat at all. At the recall-first point the `n` rows emit 7.266 false positives per image against 1.165 for the `s` rows—a $6.2\times$ difference. Precision at that point is 0.185 against 0.579. This is an example of why a deployment-oriented evaluation should report threshold selection and false alarms alongside mAP, not evidence that `n` is generally worse or that the ranking reverses across model families.

The comparison remains exploratory: validation mAP is lower for `n`, and the validation composition differs from test. A single operating point is a thin basis for any ranking, so we also swept it. For each seed we chose, on validation only, the confidence meeting a recall target and then read the test outcome (Table VII). The ranking of the two sizes is not stable across targets in any direction we can defend.

TABLE VII
OPERATING-POINT SWEEP. FOR EACH SEED THE CONFIDENCE MEETING A VALIDATION RECALL TARGET IS CHOSEN ON VALIDATION AND APPLIED UNCHANGED TO TEST; ONLY TEST PRECISION IS SHOWN. ROWS ARE SEEDS. NO TARGET PRODUCES A CONSISTENT `n`/`s` ORDERING.

model	seed	$R_{\text{val}} \geq 0.90$	$R_{\text{val}} \geq 0.95$	$R_{\text{val}} \geq 0.97$
yolov8s	0	0.975	0.854	0.396
	1	0.968	0.735	0.356
	2	0.979	0.875	0.764
	3	0.949	0.795	0.217
	4	0.964	0.862	0.731
yolov8n	0	0.931	0.685	0.186
	1	0.931	0.752	0.341
	2	0.927	0.686	0.099

Three readings of Table VII, in decreasing order of what the data supports:

(i) *At the strictest target the separation is real.* At $R_{\text{val}} \geq 0.97$ every `s` seed except one beats every `n` seed, and the single exception (`s` seed 3, 0.217) is a seed whose validation threshold collapsed to 0.004 and which therefore over-produces boxes on test. The recall-first regime used in Table VI is this column, and its direction is consistent.

(ii) *The separation is not preserved at intermediate targets.* At $R_{\text{val}} \geq 0.95$ the `s` range is 0.735–0.875 and the `n` range is 0.685–0.752. These overlap: `n` seed 1 at 0.752 sits above `s` seed 1 at 0.735, a seed whose validation threshold fell to 0.092. So the claim "`n` emits more false alarms" is true of the mean at the operating point we report and at the extremes of the sweep, but false as a statement about individual seeds at a comparable recall.

(iii) *We cannot claim a reversal either.* `n` never overtakes `s` as a group at any target, so reporting this as evidence that the smaller model wins would be equally unsupported. What the sweep shows is narrower and less quotable than either headline: the `s` seeds are better separated from `n` at $R \geq 0.90$ (0.949–0.979 vs. 0.927–0.931) than at $R \geq 0.95$, where a single badly-transferred threshold is enough to put one `n` seed above one `s` seed. The direction of the comparison is therefore stable but its margin is not, and a per-seed ranking whose magnitude depends on which recall target the user picks is

not a ranking. That is the reason we report operating points per seed rather than a single comparison, and it is why the $\pm$ entries in Table VI should not be read as a model comparison.

We note one more inconvenient detail: measured latency does not favour the smaller model (6.9 vs. 6.5 ms), because at this size the fixed per-image overhead dominates. The GFLOP and parameter savings are real, the wall-clock saving on this hardware is not. Reporting FLOPs as a proxy for latency is another summary statistic that fails to transfer.

Table VIII exposes the mechanism. The five seeds reach nearly identical test mAP (0.973–0.981) and nearly identical box recall (0.968–0.988), but the validation procedure selects thresholds spanning 0.01 to 0.21, producing precisions from 0.356 to 0.764 and FP/image from 0.40 to 2.38. Seed 2 is roughly five times more conservative than seed 1 at the same mAP while matching its recall. Any claim about "the model's precision" that pools these seeds is a claim about the threshold-selection procedure, not about the detector. We therefore report per-seed values wherever precision or FP/image is the quantity of interest, and treat the five-seed mean as a summary of protocol variance rather than of model quality.

B. Where the variance lives: source structure

The source-cluster bootstrap makes the same point from a different direction. For seed 0, resampling the three test sources gives $P \in [0.195, 0.874]$, $R \in [0.948, 1.000]$, and $\mathrm{FP/img} \in [0.139, 30.1]$. With three clusters these intervals are honest but almost uninformative; the leave-one-source-out analysis is more legible. Removing `Jan_lwir`—the ten images that generate most of the false alarms—raises seed-0 precision from 0.427 to 0.853 and drops FP/image from 1.75 to 0.17. Removing `ds0` does the opposite: precision falls to 0.246 and FP/image rises to 8.16, because the remaining `Jan` false positives are divided by far fewer true positives.

Table IX gives the direct decomposition. Ten `Jan` test images account for 301 of the 331 false positives emitted by arm A—91% of all false alarms come from 5% of the images. The two-stage arm reduces that to 21, and reduces total FP/image from 1.75 to 0.15, but it also costs recall on that source ($0.948 \rightarrow 0.909$). The honest summary is that the aggregate metric is dominated by a source that is over-represented in boxes and under-represented in images, and that any intervention which suppresses `Jan` false alarms will look like a large aggregate gain while being a modest, source-specific change.

We flag a confound: `Jan` and `May` are the sources whose filenames claim thermal imagery but whose pixels are colour photographs with marker pins (Section III). If a marker cue is present, `Jan` false alarms may be marker detections rather than mine detections, and the second stage may be suppressing markers rather than terrain. Our twelve-crop check did not find a consistent marker near test mines, but the confound cannot be excluded from this data, and it is one reason we do not claim the two-stage gain is about mines.

C. Image level

Table X asks a coarser question: does the image contain a mine at all? The frozen-backbone variant of the MobileNetV3-Large baseline is degenerate on this task. Its stage-A row reaches 0.974 accuracy but balanced accuracy 0.500 and $\kappa = 0.000$: it calls every image positive, which is correct for 184 of 189 images and useless for triage. Fine-tuning the last layers (stage B) lifts balanced accuracy to 0.562 ± 0.056 and κ to 0.156 ± 0.148. That is above chance and it is not nothing, but it is close to the noise floor: one of the three seeds still has $\kappa \approx 0$ (-0.009), and the interval over three seeds reaches down to zero agreement. We describe the frozen variant as degenerate and the fine-tuned variant as weak-but-nonzero rather than lumping both under "degenerate".

The detector is a different regime. At its validation-chosen deploy threshold it reaches balanced accuracy 0.836 ± 0.089 and $\kappa = 0.683 \pm 0.159$; at OP_{95} it reaches 0.973 ± 0.044 and $\kappa = 0.780 \pm 0.047$. The two-stage arm reaches 0.916 ± 0.084 and $\kappa = 0.767 \pm 0.120$. The high variance of the deploy row is the threshold-transfer problem again: one seed's validation choice produces a conservative test operating point and another's does not.

The frozen-classifier result is worth stating plainly because it is a negative result about a standard baseline: a classifier that achieves 97.4% accuracy has learned nothing, and accuracy on a 97.4%-positive test set is not evidence. Note that this is a property of the class balance, not of MobileNetV3-Large: with 3,887 positive and 20 negative training images the frozen features cannot recover a separating direction, whereas the fine-tuned variant does recover a weak one. We keep the frozen row to make the accuracy point, and we use the classifier only as a *second stage* over detector crops, where the proposal mechanism supplies the localisation the image-level baseline lacks and the class is balanced by construction (1:1 after crop sampling rather than 195:1).

D. Two-stage: per-seed, not pooled

Table XI is the result we most want to be read carefully. Pooled over five seeds, arm C looks like a clean win: precision 0.782 ± 0.085 versus 0.719 ± 0.046 for the recall-matched baseline, FP/image 0.377 versus 0.514. But if we treat the five seeds as the sample they are and take a confidence interval over the per-seed paired differences, every interval spans zero: $\Delta P = +0.062$ $[-0.058, +0.182]$, $\Delta\mathrm{FP/img} = -0.138$ $[-0.427, +0.152]$, $\Delta R = -0.005$ $[-0.010, +0.001]$. Pooling the arms and showing a win, then declining to pool the difference, is exactly the error this paper argues against. The per-seed structure is what the data supports.

Paired bootstrap, one row per detector seed (Table XII).

Three seeds gain, one is flat, one regresses. Seeds 0, 1 and 3 gain precision and lose a small, bounded amount of recall, with the precision and false-positive intervals excluding zero. Seed 2 is indistinguishable: the point estimates point the wrong way and every interval spans zero. Seed 4 is the case we want to state first rather than last: precision *falls* by 0.049 with an interval that excludes zero, and FP/image *rises* by 0.136 with an interval that also excludes zero. On this seed the two-stage pipeline is measurably worse than the single stage it is supposed to improve on.

TABLE VIII

PER-SEED OPERATING POINTS FOR YOLOV8S (FIVE SEEDS). THE THRESHOLD CHOSEN ON VALIDATION VARIES BY MORE THAN AN ORDER OF MAGNITUDE ACROSS SEEDS (0.01 TO 0.21) AND SO DOES THE RESULTING PRECISION. TEST MAP BARELY MOVES.

seed	val conf	test mAP_{50}	box P	box R	FP/img	image R
0	0.020	0.9741	0.427	0.976	1.751	1.000
1	0.010	0.9807	0.356	0.980	2.376	0.995
2	0.210	0.9756	0.764	0.972	0.402	0.984
3	0.045	0.9785	0.619	0.988	0.815	0.995
4	0.125	0.9726	0.729	0.968	0.481	0.989

TABLE IX

PER-SOURCE TEST RESULTS FOR SEEDS 0, ARM A (YOLO AT DEPLOY THRESHOLD) AND ARM C (TWO-STAGE). ALL 331 ARM-A FALSE POSITIVES COME FROM THREE SOURCES; 301 OF THEM FROM TEN JAN IMAGES.

arm	source	images	TP	FP	P	R	FP/img
A	Jan_lwir	10	73	301	0.195	0.948	30.100
A	ds0	151	146	21	0.874	0.986	0.139
A	ds1	28	28	9	0.757	1.000	0.321
A	all	189	247	331	0.427	0.976	1.751
C	Jan_lwir	10	70	21	0.769	0.909	2.100
C	ds0	151	146	4	0.973	0.986	0.026
C	ds1	28	28	4	0.875	1.000	0.143
C	all	189	244	29	0.894	0.964	0.153

TABLE X

IMAGE-LEVEL DECISIONS ON TEST. MNV3 A IS SEED 0; A+B IS MEAN $\pm$ STD OVER THREE SEEDS; YOLO AND TWO-STAGE ROWS ARE MEAN $\pm$ STD OVER FIVE DETECTOR SEEDS, EACH AT ITS OWN VALIDATION-CHOSEN CONFIDENCE. BALANCED ACCURACY AND κ ARE THE MEANINGFUL COLUMNS GIVEN ONLY FIVE NEGATIVE IMAGES.

system	accuracy	balanced acc	precision	recall	κ
MNv3 A (thr 0.5)	0.974	0.500	0.974	1.000	0.000
MNv3 A (recall-first)	0.968	0.497	0.973	0.995	-0.009
MNv3 A+B (thr 0.5)	0.974 ± 0.000	0.500 ± 0.000	0.974 ± 0.000	1.000 ± 0.000	0.000 ± 0.000
MNv3 A+B (recall-first)	0.968 ± 0.005	0.562 ± 0.056	0.977 ± 0.003	0.991 ± 0.006	0.156 ± 0.148
YOLO deploy	0.984 ± 0.007	0.836 ± 0.089	0.991 ± 0.005	0.992 ± 0.006	0.683 ± 0.159
YOLO OP_{95}	0.986 ± 0.003	0.973 ± 0.044	0.999 ± 0.002	0.987 ± 0.003	0.780 ± 0.047
Two-stage (arm C)	0.987 ± 0.006	0.916 ± 0.084	0.996 ± 0.005	0.991 ± 0.003	0.767 ± 0.120

Seed 4's failure has a diagnosable cause, and the diagnosis is the paper's point rather than a rescue. Its validation-only selection rule chose protection $P = 0.8$ with classifier threshold $\tau = 0.03$; at that cutoff the classifier vetoes almost nothing, so the surviving detection pool is larger than arm B's and precision drops. Validation already preferred YOLO-only at that seed — FP/image 1.448 for C against 0.956 for A, the only seed of five where this happens — so the rule had the information it needed and still selected the regressing configuration. The rule minimises FP/image subject to a 0.005 recall-drop cap, and it has no protection against choosing a near-inert classifier that satisfies the cap trivially. As a purely diagnostic check (not a candidate result, since it changes the selection rule after seeing test), forcing the stage-1 confidence to 0.125 at that seed gives test $P = 0.844$, $\text{FP/img} = 0.238$ against 0.729 and 0.481 for arm B. The classifier is not broken at seed 4; the selection rule is. We report the uncorrected number as the result and the corrected one as an explanation.

The mechanism behind the wins is visible in the arm-A confidences. At seeds 2 and 4 the validation procedure selects the *same* confidence for the deploy and matched-recall arms (0.21 and 0.125 respectively), so arms A and B coincide: the baseline is already operating at a well-transferred threshold and the classifier has little error mass to remove. At seeds 0, 1 and 3 the validation thresholds are 0.02, 0.01 and 0.045, which transfer badly to test and produce 1.75, 2.38 and 1.02 false positives per image. The classifier repairs that specific failure. It is not improving localisation; it is compensating for a threshold that did not generalise.

Two consequences follow, and both cut against the headline. First, the two-stage effect is not a stable property of the architecture; it is conditional on the baseline being badly thresholded by validation. Any paper reporting the five-seed mean without this decomposition is reporting a protocol artifact. Second, the classifier's advantage is not robustness to threshold choice. If the single-stage threshold is chosen on validation with the same freedom we allowed the pipeline, there exists a YOLO operating point that dominates it. For seed 0, a YOLO

TABLE XI

TWO-STAGE ABLATION ON TEST, MEAN ± STD OVER THE FIVE DETECTOR SEEDS 0–4 (ARM THRESHOLDS IN THE SECOND COLUMN, IN SEED ORDER). ARM A IS YOLO AT ITS DEPLOY THRESHOLD; B IS YOLO AT THE CONFIDENCE MATCHING ARM C'S VALIDATION RECALL; C ADDS THE CLASSIFIER WITH PROTECTION P.

arm	conf / τ (s0–s4)	P	R	F_2	FP/img	image R
A	0.02 / 0.01 / 0.21 / 0.045 / 0.125	0.568 ± 0.180	0.977 ± 0.008	0.839 ± 0.085	1.205 ± 0.847	0.993 ± 0.007
B	0.125 / 0.06 / 0.21 / 0.07 / 0.125	0.719 ± 0.046	0.973 ± 0.003	0.908 ± 0.013	0.514 ± 0.120	0.990 ± 0.005
C	P 0.4/0.4/0.4/0.4/0.8; τ 0.27/0.23/0.22/0.13/0.03	0.782 ± 0.085	0.968 ± 0.003	0.923 ± 0.023	0.377 ± 0.184	0.991 ± 0.003

TABLE XII

PAIRED BOOTSTRAP FOR ARM C MINUS ARM B, COMPUTED SEPARATELY PER DETECTOR SEED (5,000 RESAMPLES OVER THE 189 TEST IMAGES). AN INTERVAL SPANNING ZERO MEANS THE ARMS ARE INDISTINGUISHABLE AT THAT SEED; AN INTERVAL THAT EXCLUDES ZERO IN THE NEGATIVE DIRECTION FOR ΔP MEANS THE TWO-STAGE ARM IS WORSE.

detector seed	ΔP [95% CI]	ΔFP/img [95% CI]	ΔR [95% CI]
0	$+0.128$ $[0.065, 0.186]$	-0.253 $[-0.460, -0.095]$	-0.008 $[-0.020, 0.000]$
1	$+0.144$ $[0.096, 0.181]$	-0.347 $[-0.598, -0.159]$	-0.008 $[-0.020, 0.000]$
2	-0.037 $[-0.077, 0.013]$	$+0.093$ $[-0.021, 0.228]$	$+0.000$ $[-0.020, 0.017]$
3	$+0.126$ $[0.092, 0.163]$	-0.317 $[-0.471, -0.196]$	-0.007 $[-0.022, 0.008]$
4	-0.049 $[-0.091, -0.002]$	$+0.136$ $[0.005, 0.302]$	$+0.000$ $[-0.011, 0.010]$

confidence of 0.52 chosen on validation gives test $P = 0.976$, $R = 0.949$, $\mathrm{FP/img} = 0.032$. The best validation-chosen two-stage configuration under the same objective reaches $P = 0.945$, $R = 0.957$, $\mathrm{FP/img} = 0.074$ — 3.0 points less precision and more than twice the false alarms, for 0.8 points more recall. The two F_2 scores are within 0.0004 of each other (0.9539 against 0.9543), so this is *not* a case where one arm wins on F_2; it is a case where the whole F_2-optimal frontier has a point that a tuned single-stage threshold already reaches, and the two-stage arm merely lands on a different part of it. That is a weaker claim than "the classifier loses," and it is the one the data supports.

The narrower, defensible role for the classifier is the strict high-recall regime ($R \geq 0.96$), where the single stage's false-alarm rate grows faster than the classifier's, and terrain-nuisance rejection. It is not a general-purpose improvement, and we say so.

E. Cost of the second stage

The precision comparison above is silent on what the second stage costs, and the cost is not uniform across seeds. Table XIII reports the measured end-to-end time per image for the two-stage arm, from the same frozen test pass that produced Table XI. Each figure is wall-clock over the whole test split on one 8 GB laptop GPU, divided by image count, and covers stage-1 proposal generation, square-crop extraction with 20% context, and MobileNetV3 scoring of every proposal. It is therefore a like-for-like successor to the 6.5 ms single-stage row of Table VI, measured on the same machine.

TABLE XIII

MEASURED END-TO-END COST OF THE TWO-STAGE ARM, PER DETECTOR SEED. EACH VALUE IS WALL-CLOCK TIME ON THE FULL TEST SPLIT DIVIDED BY IMAGE COUNT, AND INCLUDES PROPOSAL GENERATION, CROP EXTRACTION AND CLASSIFIER SCORING AT A LOW PROPOSAL THRESHOLD, SO IT IS AN UPPER BOUND ON DEPLOYED COST. "CROPS" COUNTS PROPOSALS REACHING THE CLASSIFIER. PC GPU FP32, 640×640 INPUT, 224×224 CROPS.

seed	ms/img	crops/img	crops p95
0	16.01	3.06	3.6
1	17.83	3.69	6.0
2	18.09	2.76	4.8
3	15.50	2.34	6.6
4	15.19	2.35	5.6
mean	16.5 ± 1.4	2.8 ± 0.6	5.3 ± 1.4

Three things follow. First, the second stage costs roughly $2.5\times$ the single-stage detector (16.5 against 6.5 ms), and unlike the model-size comparison in Section VII-A this ordering is stable: every seed is slower than the single stage, and the seed-to-seed spread (1.4 ms) is small relative to the penalty (10 ms). Second, the cost tracks proposal count rather than any model property, which is why the two seeds with the lowest crop counts (3 and 4, at 2.34 and 2.35) are also the two fastest; nothing about the classifier's accuracy predicts its cost. Third, the tail is worse than the mean: the 95th percentile of crops per image is 5.3 across seeds against 3.6 at seed 0 alone, so a number quoted from one seed understates exactly the frames that matter for a fixed-rate pipeline. Cost is a property of the whole detector–classifier configuration, and it must be reported per seed like every other quantity here.

A final caveat on the unit of replication. The five seeds are not five fully independent end-to-end systems. Seeds 0, 1 and 2 train their stage-2 classifier on the same crop set, generated once from the seed-0 detector (`paper_results/stage2/crops/`); seeds 3 and 4 generate their own crops from their own stage-1 checkpoints. At *evaluation* time all five runs are detector-aligned: each scores its own detector's detections, which is why the seed-1 and seed-2 runs are labelled `s1_aligned` and `s2_aligned` and why their arm-A operating points (0.01 and 0.21) coincide with those seeds' own validation-chosen confidences rather than

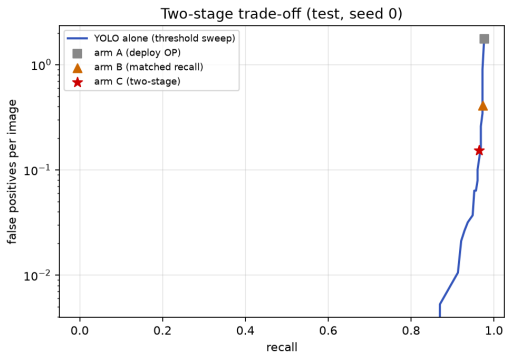


Fig. 3. False positives per image against recall for the single-stage and two-stage arms (seed 0). The classifier's advantage is concentrated at high recall; at moderate recall a tuned single-stage threshold is better. Curves are drawn from the same frozen test records used in Table XI.

seed 0's 0.02. Each run also uses a separately trained stage-2 classifier (`s2_ratio1_s0` through `s4`, distinct checkpoints with distinct file hashes), so the classifier side is not shared. What is shared is the stage-2 *recipe* throughout, and the crop-generation detector for seeds 0–2 only. The effective number of independent replications is therefore closer to four than five, and the $\pm$ standard deviations in Table XI should be read as variation across threshold-selection outcomes, not as variation across independently trained end-to-end systems.

F. Second-stage ablations

Five variants isolate the recipe (Table XIV). Removing hard negatives is the largest single degradation ($\Delta P = -0.056$ $[-0.084, -0.028]$), which is unsurprising given that the negatives are detector false positives rather than random background. Removing context padding (0.0) costs 0.027 precision, and doubling it (0.4) costs 0.025: among the three padding values tried, 20% is the best observed value; this does not establish a general optimum. Reducing crop resolution to 160 *improves* precision by 0.017 with an interval that touches zero $[0.000, 0.036]$, a difference we do not claim. Doubling to 320 costs 0.025. All intervals are paired image bootstraps against the baseline on the same test images.

G. Grad-CAM: evidence, not correctness

We use attribution as a falsification tool and report what it shows, including where it is uninformative. The headline aggregate on 244 true-positive detections is that the median share of CAM energy inside the ground-truth box is 0.939 (mean 0.690, first quartile 0.0002, third quartile 0.996), median enrichment over the box's area share is 58.2×, and the peak falls inside the box for 73.0% of detections. That aggregate is misleading on its own, and we do not report it on its own for the rest of this section. Figure 4 shows the distribution it summarises, and why a median is the wrong summary for it.

Stratifying the same detections by how many ground-truth boxes the image contains changes the picture completely. The test split has 172 single-object images and 12 multi-object images (253 boxes in 189 files). On the 170 single-object detections the median energy share is 0.977 (`s` seed 0; 0.988 and 0.981 for seeds 3 and 4), the mean share is 0.90–0.93, and median enrichment is 88–91×: the detector looks at the object. On the 74 multi-object detections the median share is 0.000 and enrichment 0.000, with 62 of 74 detections below 0.01; the recurrence of the six multi-mine `Jan` frames is what the first-quartile figure of 0.0002 actually measures.

This is a metric artifact, not a model failure. The E1 implementation computes one CAM per image from the single top-scoring anchor, then attributes every detection on that image to the same map; on a ten-mine frame, nine correctly localised detections are scored against a map built for the tenth. We therefore report the single-object stratum as the informative one, stay silent on the multi-object stratum, and explicitly decline the interpretation this pattern would otherwise invite — that a bimodal energy distribution means detections are "right for the wrong reason." The data cannot separate a genuine low-attribution detection from an aggregation bug, and since the low-energy mass is entirely inside the stratum where the bug applies, that interpretation is untestable here.

Deletion tests confirm the direction without establishing a mechanism. Masking the ground-truth box lowers the detection score by 0.739 on average ($n = 182$); masking a matched background region lowers it by 0.0015. The detector's score depends on the object region, as it should. It does not follow that the detector's score depends *on context*, and nothing in this section tests that.

The head-to-head comparison is likewise a statement about where the two models look, not about why either is right. On images where both the detector and the image-level classifier are correct, detector enrichment has median 86.0× and the MobileNetV3-Large baseline has median 1.82×; the paired difference is $+147.0$ $[125.4, 170.0]$ in favour of the detector. On detector crops, the stage-2 classifier reaches 2.13×. Neither of the smaller figures should be read as a failure: with 20% crop padding the box occupies most of the crop, so enrichment is bounded near $1/0.7^2 \approx 2$ by geometry alone, and the image-level baseline predicts a bounding box for the whole frame, for which "inside the box" is not a meaningful target. The image-level baseline is also degenerate on negatives in the image-level task ($\kappa \approx 0$). These measurements do not establish context exploitation and do not explain the two-stage effect.

A paste-and-remove test probes the same question behaviourally. Pasting a mine crop into a background yields detection at 0.650 under the deploy confidence and 0.630 at the matched-recall confidence, against 0.976 recall on original images; inpainting a mine out of an image makes it disappear in 100 of 100 composites. The asymmetry is expected: synthetic pastes carry feathered edges and scale mismatch, so the detector is being asked to fire on an object that looks slightly wrong. We report the numbers as an upper bound on the paste setup's realism, not as an estimate of the detector's sensitivity to context.

The one zero-energy result we do stand behind is at the image level, not the detection level. The E5 flag pass aggregates energy over the *union* of all ground-truth boxes on an image,

TABLE XIV

STAGE-2 ABLATIONS AT THE VALIDATION-CHOSEN RECALL-SAFE RULE, DETECTOR SEED 0. ΔP IS A PAIRED IMAGE BOOTSTRAP AGAINST THE BASELINE; AN INTERVAL SPANNING ZERO MEANS INDISTINGUISHABLE.

variant	pad	crop	hard neg.	τ	P	FP/img	ΔP [95% CI]
baseline	0.2	224	yes	0.27	0.894	0.153	reference
crop 160	0.2	160	yes	0.43	0.911	0.127	$+0.017$ $[0.000, 0.036]$
crop 320	0.2	320	yes	0.08	0.868	0.196	-0.025 $[-0.049, -0.006]$
no hard neg.	0.2	224	no	0.35	0.837	0.254	-0.056 $[-0.084, -0.028]$
pad 0.0	0.0	224	yes	0.16	0.866	0.201	-0.027 $[-0.043, -0.011]$
pad 0.4	0.4	224	yes	0.25	0.868	0.196	-0.025 $[-0.042, -0.011]$

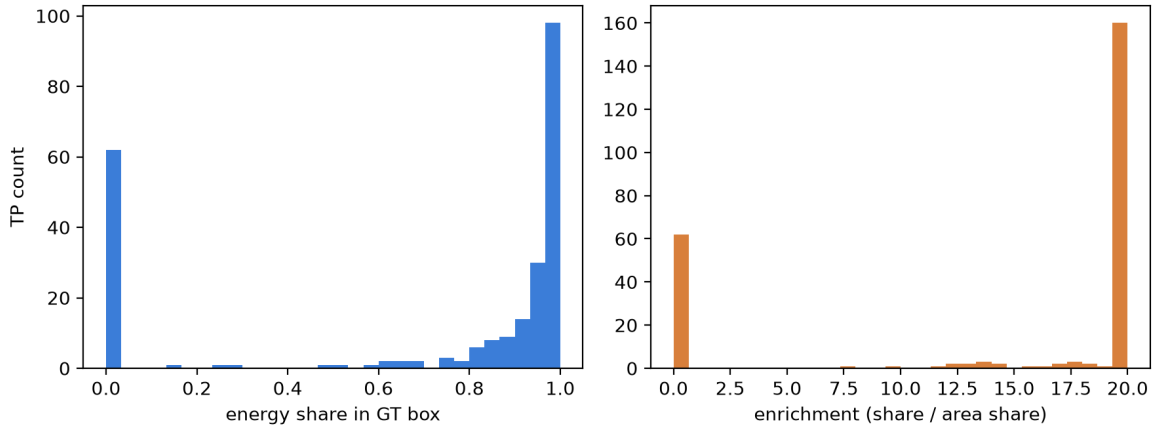


Fig. 4. The distribution behind the aggregate in the text, shown because the aggregate hides it. Left: share of CAM energy falling inside the ground-truth box, $n = 244$ true-positive detections. Right: enrichment, the energy share divided by the box's share of the image area, clipped at $20\times$ for display. The distribution is strongly bimodal, and the low mode is populated entirely by detections on multi-object frames, where E1 attributes every detection on an image to one CAM. We show the histogram rather than only the median because the median (0.939) sits between the two modes and describes neither.

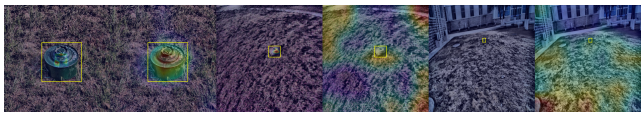


Fig. 5. Grad-CAM on detector crops (top) and image-level classifier (bottom), all from single-object images. On such images the detector's evidence is object-localised; the classifier's is diffuse. On multi-object images the E1 aggregation reuses one per-image CAM for every detection, so detector panels are not shown for that stratum. This is a statement about where each model looks, not about whether either prediction is correct.

which sidesteps the per-image CAM problem above. Of 184 true-positive test images, 19 fall below the 0.800 threshold, and hand review of those found exactly two — `ds0_0801` and `ds0_0883` — whose annotations are objectively wrong, one missed mine each. This is consistent with the attribution pipeline tracking something real at the image level, but it is a hypothesis about two images, not a validated audit method: we did not independently re-derive the label errors by any other route, and we make no claim about the remaining 17 flagged images.

Finally, the failure catalogue (Table XV) is the qualitative counterpart. Of 29 reviewed false positives, 22 are terrain texture; of 9 false negatives, 5 are camouflaged or low-contrast objects and 2 are small. The failure mass is where the deployment risk is, and it is not visible in any aggregate score.

TABLE XV

HAND-REVIEWED FAILURE CATEGORIES AT THE ARM-C DEPLOYMENT THRESHOLD (SEED 0). FALSE POSITIVES ARE CAPPED AT 50; 29 WERE PRESENT.

category	FN	FP
camouflaged or low contrast	5	0
small object	2	0
label error	2	0
terrain texture	0	22
low-light ground	0	2
fence / structure edge	0	2
structure wall	0	1
sensor noise	0	1
tree trunk	0	1
total	9	29

VIII. DISCUSSION

Report the operating point, then the curve.: The size sweep is a case study showing that mAP and deployment behaviour are different measurements. Three `yolov8n` seeds and five `yolov8s` seeds are close on test mAP (0.979 ± 0.003 vs. 0.976 ± 0.003, intervals overlapping), but validation mAP

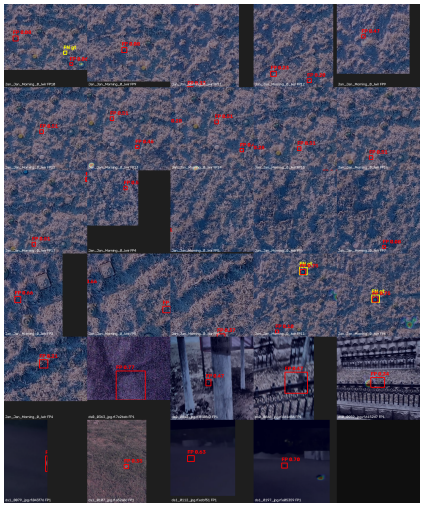


Fig. 6. Representative false positives. Terrain texture dominates; the distinction between a mine-like disc and a rock or a patch of ground is the failure mode that matters operationally.

favors s by more than a seed standard deviation and the two families differ by $6.2\times$ in false alarms at the recall-first point. The result should motivate a threshold-rule sweep rather than a model ranking claim. The mechanism is not mysterious: mAP integrates over confidence, and a model can rank detections well while being badly calibrated near the threshold a user will pick. The recommendation is concrete and cheap. Report, at minimum: the validation procedure used to choose a threshold, the threshold itself for every seed, and precision, recall, and false-positives-per-image at that threshold on test. Report mAP as well, for comparability, but do not let it be the only row.

Threshold transfer is the hidden variable.: Our five seeds select thresholds of 0.02, 0.01, 0.21, 0.045 and 0.125 for the same architecture, same data, same schedule. That is not a bug in the search; it is what happens when a validation set has 181 images and 7 negatives. The consequence is that "model precision" pooled over seeds is mostly a statement about which threshold each seed happened to pick. This is consistent with the two-stage result: the classifier looks valuable exactly where the threshold transferred badly, and actively harmful where the selection rule chose a near-inert configuration, but five seeds with shared components do not identify that as a mechanism. Any ablation that changes a component downstream of a fragile threshold will inherit that fragility, and a single pooled mean will hide it.

When is a second stage worth it?: On this evidence, narrowly. The classifier is defensible if the requirement is recall ≥ 0.96 with minimum false alarms, and if the detector's threshold is known to be unreliable. It is not defensible as a general precision improvement, because a tuned single-stage threshold beats it at max-F_2, and its benefit disappears at the seed whose threshold transferred well. The cost is real and it is not a rounding error: 16.5 ± 1.4 ms per image end to end against 6.5 ms for the single stage, with 2.8 ± 0.6 crops scored per image and a 95th percentile of 5.3 (Table XIII). For a pipeline that processes every frame of a flight, that is a $2.5\times$ multiplier bought for a gain that is conditional on the detector's threshold being unreliable. A deployment decision here is not "which model is more accurate" but "how many false alarms is worth $2.5\times$ the frame budget", and only the second question has an operational answer.

Leakage is worth measuring, not just fixing.: The contamination-free comparison is a negative result, and we think negative results of this kind should be routine. Fixing a split is good practice; quantifying what the fix changed is what tells the reader whether the original numbers were wrong by 0.002 or by 0.2. Here it is 0.0003 on the comparable subset. We would not have known that from the audit alone, and we would have been tempted to describe the leak as catastrophic.

What the saliency audit bought.: Not much, and we say so rather than dressing it up. On the 12 multi-object test images the metric we used is not even well defined, because a single per-image CAM is reused for every detection; on the 172 single-object images the detector's evidence is cleanly object-localised (median energy share 0.98, enrichment $\approx 90\times$), which is a statement about localisation, not about correctness. The image-level baseline's low enrichment is partly a promise of the 20% padding geometry under which the ground-truth box occupies most of the crop, and the frozen variant is degenerate on negatives. The audit's one useful output was as a consistency check on two manually identified label errors. Attribution did not prove any prediction correct, and the revised interpretation keeps it as a sanity check rather than a mechanistic explanation.

IX. System Design and the Provenance Gap

The preceding sections evaluate models. A deployed system is a different object, and the distance between the two is where most claims about this work must stop. We describe what the accompanying deployment project (AeroShield) contains, and then report a finding that belongs to this paper's central argument even though it concerns engineering rather than detection.

Architecture, as design and not as measurement.: The project targets a Jetson Nano (4 GB, Maxwell generation, JetPack 4.6 / L4T 32.7.6, CUDA 10.2, TensorRT 8.2, Python 3.6). Inference is onboard and the ground station is a client. A FastAPI service backed by a PostGIS store handles missions, detections, operator adjudication and report generation, with a web frontend providing a map view, a Grad-CAM comparison and an adjudication workflow.

Two features of this design matter for how the present results should be read.

First, *the backend runs no model*. Its image-upload detection endpoint is a development stub returning a fixed detection with a fixed confidence, and the endpoint's own docstring cites the project requirements (§2.2) as ruling out server-side inference. This is the right architecture for the task—the sensor is on the aircraft, and the ground station's job is bookkeeping, human review and reporting—but it has a consequence for reporting: no throughput, latency or accuracy figure obtained from the backend describes detection performance. A system

TABLE XVI

WHAT WAS EVALUATED AGAINST WHAT WAS DEPLOYED. THE RIGHT-HAND COLUMN IS ASSEMBLED FROM THE DEPLOYMENT PROJECT'S TRAINING SCRIPT, ITS COMMITTED REPORT AND THE SIZE OF THE ONNX FILE IT SHIPS; THE THREE AGREE WITH EACH OTHER. NEITHER REPOSITORY RECORDS WHICH FILE REACHED THE DEVICE.

	evaluated here	deployment artifacts
backbone	YOLOv8s	YOLOv8m
parameters	11.2 M	25.9 M
classes	1 (`Landmine`)	1 (`Landmine`)
split	v2, 3,907/181/189	v1, 5,723/455/221
leak-audited	yes, 30 removed	no
provenance	manifest digest	not recorded
evaluated here	—	no

diagram that shows a detector on both sides of the radio link is architecturally wrong as well as unsupported.

Second, *the export path is constrained by the device*. The detector is exported to ONNX at a fixed 640×640 input with opset 12 and non-maximum suppression left out of the graph, then compiled on the target with `trtexec --fp16`. The opset ceiling is set by the platform, not by preference: JetPack 4.6 ships TensorRT 8.2, and an engine produced for a newer TensorRT cannot be loaded. The engine must be built on the device; an engine compiled on a desktop GPU will not run on Maxwell. The ONNX file is portable and the engine is not, which is a common source of edge numbers that cannot be reproduced from the repository that reports them.

The provenance gap.: Our evaluation and the deployment diverged, and neither repository records where. The models evaluated here are YOLOv8s (11.2 M parameters) and YOLOv8n (3.0 M) trained on the rebuilt split $3{,}907/181/189$ and evaluated on a leak-audited test set. The deployment project's committed evidence points elsewhere: its training script sets the base model to `yolov8m.pt`, its committed training report documents a 75-epoch YOLOv8m run on the pre-rebuild split ($5{,}723/455/221$), and the ONNX file it ships is 103.6 MB beside a 42.7 MB PyTorch checkpoint of the same name. The parameter ratio between the two backbones is $25.9/11.2 = 2.31$; the file-size ratio is $103.6/42.7 = 2.32$. Both artifacts declare one class (`Landmine`), so this is not the class-count confusion we corrected earlier in this project's own documentation—it is a question of which backbone and which split. Which file was actually flashed to the device is not recorded in either repository.

This is a leakage failure of a different kind, and it is worth naming precisely, because the vocabulary of the field does not cover it. The literature on leakage describes *data* crossing a split boundary. What happened here is an *artifact* crossing a project boundary: a checkpoint was produced, reported, and deployed, and the report describing it was filed in a directory that was later emptied by a rebuild. Every individual step was reasonable. The training run is documented; the report is committed; the weights directory has a README explaining how to regenerate the checkpoint. The failure is compositional. The only link between the evaluated model and the deployed model was a filename, and the filename was reused.

The methodological recommendation is narrow and cheap. A commit hash identifies the *code* that produced a model, not the model. A checkpoint manifest—file name, SHA-256, backbone, class list, split digest, and the export parameters used to produce the ONNX—identifies the model, and it can be verified from the artifact in a few seconds without re-running anything. This paper ships such a manifest for the evaluated models (Section XII). The deployment side does not, and the cost of that absence is exactly the ambiguity in Table XVI: a defensible claim that a detector of this family runs on this hardware, and no defensible claim about which one.

What remains unmeasured.: The deployment repository contains no committed timing record. Its edge numbers appear in prose in its README, no run artifacts are stored, and therefore none of them can be verified, re-derived, or compared with Table VI. We do not import them. The gap is not a contradiction to be argued with but an absence to be filled: the models evaluated here are exported and ready to run onboard, and producing an on-device number for one of them is an instrumentation task—build the engine on the device, run the existing per-stage timing harness, commit its summary—rather than a change of method. Until that is done this paper makes no claim about edge throughput, energy, or thermal behaviour, and the deployment project's architecture should be read as design intent rather than as a validated result.

X. LIMITATIONS

1) **Compute.** All results come from one 8 GB laptop GPU and roughly 30 GPU-hours. This single constraint explains most of the gaps below. We state it once rather than repeating it as a caveat on every claim.
2) **One detector family, two depths on the clean split.** YOLOv8s (five seeds) and YOLOv8n (three seeds). YOLOv8m was trained only on the old leaky split and is therefore not comparable; we exclude it from the size sweep rather than presenting a contaminated point.
3) **Small-model seed count, and shared components.** The `yolov8n` row now carries an interval over three seeds, but its test mAP sits inside the `yolov8s` interval, so the two are not distinguishable on that metric. The five two-stage seeds are also not five independent systems. All five are detector-aligned at evaluation time, each scoring its own stage-1 detections, but seeds 0–2 train their classifier on a single crop set generated once from the seed-0 detector, while seeds 3–4 generate their own crops. The stage-2 recipe is held fixed throughout, and the stage-2 classifiers themselves are separately trained (distinct checkpoints per seed). The effective replication is closer to four.
4) **Unresolved checks.** We did not sweep recall targets and FP budgets exhaustively, did not threshold-match the leaky and clean models, and did not retrain after the label correction. The leak count rests on the operational filter plus an embedding-neighbour check that agreed on all 99 survivors but cannot exclude a single missed near-duplicate; one candidate pair (`ds1_0054`/`ds1_0111`) is reported in the run log as unconfirmed.

5) **Five negative test images.** Precision and FP/image are high-variance; the source-cluster intervals span an order of magnitude. Point estimates in tables are accompanied by intervals wherever the interval is meaningful.
6) **Label correction without retraining.** Two test boxes were removed from ground truth. We report both versions, but the model was not retrained on corrected labels because no training-label errors were found.
7) **No grouped cross-validation.** A single frozen split was used, as required by the protocol. Group-aware k-fold would give a better estimate of split variance and was not affordable.
8) **No hard-negative re-mining on unseen data.** The stage-2 negatives come from detector false positives on the training split, which is the correct procedure but limits how novel the negatives are.
9) **Synthetic composites.** The paste-and-remove experiment uses feathered, scale-mismatched composites. It is a probe, not a counterfactual.
10) **No on-device characterisation.** All latency numbers in Table VI are PC GPU FP32 at 640×640. No edge measurement is reported here, so we make no claim about Jetson throughput or energy. The accompanying deployment project contains no committed timing record, so its own prose figures cannot be verified or reused; the artifact mismatch behind that gap is described in Section IX. Obtaining an on-device number for the models above is an instrumentation and re-export task, not a change of method.
11) **Source confound.** `Jan` and `May` are colour photographs with marker pins, despite `lwir` in the filename. Marker shortcuts cannot be fully excluded, and the source dominates aggregate false alarms.
12) **No field validation and no buried-object claim.** This is a benchmark on visible surface indicators. It says nothing about burial depth, soil type, or detection probability in real clearance conditions.

XI. Ethics and Dual Use

What this system can and cannot do.: The detector recognises visible mine-like objects and surface indicators in downward-looking colour imagery. It cannot detect a buried object, because the sensor cannot see one. It has not been flown, field-tested, or validated against ground truth collected in a real clearance operation. Nothing in this paper supports a statement about detection probability in the field, and we do not make one. The correct description is a computer-vision benchmark on a public, publicly derived image corpus.

Dual use.: The same capability that helps a clearance team prioritise ground could help someone avoid marked hazards. We publish standard detection metrics, an audit methodology, and trained weights; we publish no targeting guidance, no geographic coordinates, and no operational deployment procedure. The dual-use risk is real but generic to visible-object detection, and we judge that the methodological contribution—how to audit a leaky split and why to report operating points—outweighs it. We note that the weights are released under AGPL-3.0, which requires derivative services to publish their source; this is a licence choice, not an ethical safeguard.

The cost of the miss rate.: At the primary operating point the two-stage arm recalls 0.968 ± 0.003 of annotated instances over the five detector seeds (0.964 at seed 0, the run whose failures we analyse), which on this test set means nine missed objects out of 253. Those nine are analysed in Table XV: five are camouflaged or low-contrast, two are small objects, and two are the objectively wrong labels removed under D023. At seed 0 a pre-correction recall of 0.964 reads as a good number in a table; after the two label corrections, the corresponding value is 0.972. In a clearance context it means that roughly one in twenty-eight annotated objects is not flagged, and the misses are concentrated in exactly the low-contrast cases where a human reviewer is most likely to defer to the model. No metric in this paper should be read as saying the system is safe to rely on.

Data ethics.: The corpus is derived from public dataset exports and contains no human subjects, no personal data, and no imagery of identified individuals. We did not collect new data. The original licences were not recorded at download time; we therefore redistribute no images (Section III). Where figures include source imagery, they are reproduced for scholarly comment at low resolution and are not a dataset release.

Reporting.: We report the results that weaken our own claims—the seed-4 regression in the two-stage pipeline (and the fact that only three of five seeds gain), the negative leakage result, the degenerate image-level baseline, the tuned single-stage threshold that beats the pipeline—in the main text rather than in an appendix. The reason is practical: a reader who is deciding whether to trust the positive results needs to know how the negative ones were handled.

XII. Availability and Licensing

Code and artifacts.: Training, evaluation, auditing, Grad-CAM and table-generation code are released with the paper, together with all evaluation JSONs, the generated tables, and the figures. The frozen split manifest is released as a CSV with the following SHA-256 digest:

```
748322d62ace769445d0bd2363699097
0c7a5115a99c9344e139099c608c80de
```

No image, label file, or crop is redistributed; see `DATA_PROVENANCE.md` for the reason and for the exact source inventory.

Weights.: Three detector seeds are released, each with `best.pt`, `last.pt` and an ONNX export, plus three stage-2 checkpoints and the original v1 run for reference, all with a SHA-256 manifest. The seed-3/4 `yolov8s` and seed-0–2 `yolov8n` runs discussed in Section VII were trained on the same frozen split under the same schedule; their evaluation JSONs are committed so every number in this paper is reproducible, but their weight files are not part of the released tree. The detector checkpoints are derivative works of Ultralytics

YOLOv8 and are distributed under **AGPL-3.0**; the stage-2 checkpoints are fine-tuned torchvision MobileNetV3-Large and are distributed under the same repository licence for consistency. Third-party terms are stated in `models/NOTICE.md`. The repository code and the audit artifacts are AGPL-3.0.

Reproduction.: Regenerating every table requires no GPU and no dataset: the committed JSONs are sufficient. The table script exits with an error if a required field is absent rather than substituting a value, so a clean clone either reproduces the tables in this paper or fails visibly.

Persistent identifier.: No DOI has been assigned at the time of this draft. The manifest digest above is a content-level identifier; a Zenodo DOI and exact release tag will be added to the camera-ready release.

REFERENCES

[1] R. Siegel, "Land mine detection," *IEEE Instrumentation & Measurement Magazine*, vol. 5, no. 3, pp. 22–28, 2002.

[2] H. Kasban, O. Zahran, S. M. S. Elaraby, and H. Osman, "A comparative study of landmine detection techniques," *Sensing and Imaging*, vol. 11, pp. 89–112, 2010.

[3] International Campaign to Ban Landmines – Cluster Munition Coalition, "Landmine monitor 2023," https://the-monitor.org, 2023.

[4] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," https://github.com/ultralytics/ultralytics, 2023, version 8.4.117, AGPL-3.0.

[6] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: A simple and strong anchor-free object detector," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[7] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European Conference on Computer Vision (ECCV)*, 2020.

[8] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *IEEE International Conference on Computer Vision (ICCV)*, 2017.

[11] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan *et al.*, "Searching for MobileNetV3," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[12] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[13] S. Kapoor, E. Cantrell, K. Peng, T. Pham, C. A. Bail, O. E. Gundersen, J. M. Hofman, J. Hullman, M. A. Lones, M. M. Malik *et al.*, "REFORMS: Consensus-based recommendations for machine-learning-based science," *Science Advances*, vol. 10, no. 18, p. eadk3452, 2024.

[14] M. Roberts, D. Driggs, M. Thorpe, J. Gilbey, M. Yeung, S. Ursprung, A. I. Aviles-Rivero, C. Etmann, C. McCague, L. Beer *et al.*, "Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans," *Nature Machine Intelligence*, vol. 3, pp. 199–217, 2021.

[15] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do ImageNet classifiers generalize to ImageNet?" in *International Conference on Machine Learning (ICML)*, 2019.

[16] B. Barz and J. Denzler, "Do we train on test data? purging CIFAR of near-duplicates," *Journal of Imaging*, vol. 6, no. 6, p. 41, 2020.

[17] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *International Joint Conference on Artificial Intelligence (IJCAI)*, 1995.

[18] C. Zauner, "Implementation and benchmarking of perceptual image hash functions," Master's thesis, Upper Austria University of Applied Sciences, 2010.

[19] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[20] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing error in object detectors," in *European Conference on Computer Vision (ECCV)*, 2012.

[21] P. Dollár, C. Wojek, B. Schiele, and P. Perona, "Pedestrian detection: An evaluation of the state of the art," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 4, pp. 743–761, 2012.

[22] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[23] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[24] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[25] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *ACM Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019.

[26] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *IEEE International Conference on Computer Vision (ICCV)*, 2017.

[27] A. Chattopadhyay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-CAM++: Improved visual expla-

nations for deep convolutional networks," in *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2018.

[28] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[29] W. Samek, A. Binder, G. Montavon, S. Lapuschkin, and K.-R. Müller, "Evaluating the visualization of what a deep neural network has learned," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 11, pp. 2660–2673, 2017.

[30] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K.-R. Müller, "Unmasking Clever Hans predictors and assessing what machines really learn," *Nature Communications*, vol. 10, p. 1096, 2019.

APPENDIX A
DECISION LOG (ABRIDGED)

The project kept a numbered decision register. The entries that bear on claims in this paper are reproduced here; the full register, with dates and rationale for all 23 entries, is in `DECISIONS.md`.

APPENDIX B
TEST-TOUCH ACCOUNTING

Eighteen numbered test touches were logged during the original run; a further seven were logged during the extended analysis. Grouped by purpose:

Only touch #4 could contaminate a claim, and it is the evaluation the paper's detector numbers come from. Touch #18 is a correction: the original seed-1/2 two-stage runs had reused the seed-0 detector, which we detected from byte-identical arm-A/arm-B records and corrected by re-running with each seed's own checkpoint. No threshold was re-chosen on test; the per-seed validation selection was rerun for the corrected detector records. Touches #19–#24 repeat the same protocol on additional seeds; none of them altered a rule, a threshold grid, or a selection criterion after looking at test.

APPENDIX C
FAILURE GALLERY AND REPRODUCTION COMMANDS

The false-positive contact sheet is reproduced in Figure 6. The full false-negative gallery, the audit montages, the per-image predictions, and the Grad-CAM heatmaps are in the repository under `paper_results/`.

Environment: Python 3.11.9, Ultralytics 8.4.117, PyTorch 2.6.0+cu124, torchvision 0.21.0+cu124, NVIDIA RTX 4070 Laptop (8 GB). Evaluation at 640×640, NMS IoU 0.7, `max_det` 300, FP32, no TTA.

TABLE XVII
ABRIDGED DECISION REGISTER. ENTRIES OMITTED HERE CONCERN FILE LAYOUT AND TOOLING RATHER THAN CLAIMS.

ID	Decision and reason
D010	Reject the original split. 76 cross-split base-name groups; 63.8% of evaluation images with a pHash-≤ 2 training neighbour; augmentation before splitting confirmed in the upstream scripts.
D011	Image-level and FP/image metrics blocked until the test split contains negatives. It contained zero.
D012	`May`/`Jan` files are colour RGB with marker pins, not thermal, despite the `lwir` filename. Keep and disclose (D016).
D014	Operational leak definition: exact MD5, or base key, or same-source pHash≤ 2 *and* 32×32 RGB MSE ≤ 900. Manual review of all 146 candidates sets the MSE cut (same-scene ≤ 817, texture ≥ 958.6).
D015	Freeze `landmine_combined_v2` at 3,907/181/189 with 30 eval leaks removed; manifest digest `748322d6...c80de`; true leak rate $0/370$.
D017	Disclosed protocol deviation: the upstream training script auto-evaluates test at confidence 0.25. No selection uses those numbers; the frozen-protocol evaluation is a separate single pass.
D018	Accept early stopping at epoch $67/75$ (best 47) for the pilot run; retrain at batch 8 for comparability (D019). The pilot is not a paper result.
D020	Frozen-feature MNv3 is degenerate on negatives at every resolution and batch size ($\kappa \approx 0$); only fine-tuning the last layers separates them. Stage A rows retained as the mandated ablation.
D021	Threshold rule: sweep protection P, choose the largest τ within 0.5 points of stage-1 validation recall, then minimise validation FP/image. Four of five seeds select $P = 0.4$; seed 4 selects $P = 0.8$.
D022	Grad-CAM flags images by energy in the union of all ground-truth boxes, not per detection; per-detection rows were an instance-count artefact on multi-mine frames.
D023	Remove two objectively wrong test boxes (tree trunk, fence); report before/after; do not retrain (no training-label errors found).

TABLE XVIII
TEST-TOUCH ACCOUNTING, GROUPED BY PURPOSE.

touches	purpose	sanctioning rule
#1–#2	audit read-only passes	leakage audit reads labels only
#3	MNv3 baseline, seed 0	Phase 4 protocol eval, val-chosen thresholds
#4	YOLO seed-0 frozen-protocol eval	the single protocol-governing eval
#5	MNv3 seeds 1–2	Phase 4 protocol eval
#6–#8	two-stage seeds 0–2	Phase 5 protocol eval, val-frozen
#9–#12	Grad-CAM E1–E6	Phase 6, read-only attribution
#13–#14	YOLO seeds 1–2	Phase 3 per-seed protocol eval
#15–#16	leaky baseline, v1 split and clean subset	extended analysis
#17	YOLOv8n size point	size ablation, val-chosen threshold
#18	corrected detector-aligned two-stage	re-score, no selection
#19	YOLOv8n seeds 1–2 and YOLOv8s seeds 3–4	seed extension, val-chosen thresholds
#20	operating-point sweep, five seeds per size	re-read of committed detection records, no selection
#21	stratified Grad-CAM re-cut	read-only attribution, strata fixed in advance
#22	threshold-selection bootstrap	re-read; validates the val-only rule, chooses nothing
#23	five-seed stage-2 audit	pipeline rerun with each seed's own detector
#24	seed-3/4 two-stage pipeline	extended analysis, val-chosen configuration
#25	embedding near-duplicate check	read-only audit, no detector evaluated

```
# tables T1-T6 and figures (no GPU, no dataset required)
python src/reporting/make_tables.py

# frozen-protocol evaluation of a checkpoint
python src/evaluation/phase3_eval.py --run <run_dir> --out <out_dir>

# two-stage pipeline with each detector seed
python src/pipeline/phase5_pipeline.py --yolo <seed_best.pt> \
    --cls <stage2_best.pt> --arm-a-conf <val_conf> --out <out_dir>

# statistical analysis, bootstrap and policy rules
python src/analysis/analysis_stats.py
```